

EVIDENCIA DE APRENDIZAJE

Simulación de Flujo CI/CD con Jenkins, Git y Automatización de Despliegues

Aprendiz:

Carlos Julio Cadena

Formación:

Integración Continua y Automatización con Jenkins

Fecha:

16 de mayo de 2026

1. Introducción

En esta actividad se realizó una simulación práctica de un flujo completo de integración continua utilizando Jenkins, enfocada en la gestión de cambios, control de versiones y automatización del ciclo de despliegue.

A diferencia de actividades anteriores, en este ejercicio se simuló un escenario más cercano a un entorno real de desarrollo, incluyendo conflictos de código, correcciones iterativas, validación de pipeline y despliegues progresivos.

2. Objetivos

Objetivo General

Simular un entorno de integración continua con control de errores, múltiples cambios y validación automatizada del pipeline.

Objetivos Específicos

Simular cambios progresivos en un proyecto.

Generar errores controlados para análisis del pipeline.

Validar comportamiento de Jenkins ante fallos.

Ejecutar correcciones y reintentos de build.

Integrar flujo de trabajo con ramas y merges.

3. Escenario de Simulación

Se parte de un proyecto en GitHub con estructura básica y pipeline configurado en Jenkins.

Se definió el siguiente escenario:

Rama principal: main

Rama de desarrollo: dev

Ramas de simulación: feature/*

El objetivo fue simular un equipo de desarrollo con múltiples cambios simultáneos.

4. Simulación de Cambios (Flujo Realista)

Iteración 1 – Cambio funcional básico

Se crea rama: feature/login-ui

Se modifica interfaz de acceso

Commit y push al repositorio

Jenkins ejecuta pipeline automáticamente

Resultado: SUCCESS

Iteración 2 – Cambio con error intencional

Rama: feature/auth-error

Se introduce error en lógica de autenticación

Push a GitHub activa Jenkins

Resultado:

Build FAILED

Error detectado en pruebas automatizadas

Notificación enviada

Iteración 3 – Corrección del error

Se corrige el bug en la misma rama

Nuevo commit y push

Jenkins ejecuta nuevamente pipeline

Resultado: SUCCESS

Iteración 4 – Conflicto de merge simulado

Merge de feature/login-ui y feature/auth-error

Se genera conflicto de código

Resolución manual del conflicto

Push a rama dev

Jenkins ejecuta validación completa

Resultado:

Build UNSTABLE (advertencias en pruebas)

Iteración 5 – Integración final controlada

Merge de todas las features a dev

Validación final en pipeline

Ejecución completa sin errores críticos

Resultado: SUCCESS

5. Comportamiento del Pipeline

El pipeline en Jenkins ejecutó automáticamente las siguientes etapas:

Clonación del repositorio

Instalación de dependencias del framework web

Ejecución de pruebas unitarias
Validación de calidad del código
Generación de estado del build
6. Gestión de estados en Jenkins

Durante la simulación se evidenciaron los siguientes estados:

Estado	Descripción
SUCCESS	Ejecución correcta
FAILED	Error en código o pruebas
UNSTABLE	Advertencias o conflictos menores

7. Notificaciones automáticas

Se activaron notificaciones en tiempo real desde Jenkins hacia:

Correo electrónico (SMTP)
Discord
Microsoft Teams
Telegram (bot automatizado)

Cada estado del pipeline generó alertas inmediatas.

8. Webhooks de automatización

Se configuró integración con GitHub mediante Webhooks para activar Jenkins en:

Push de código
Merge de ramas
Creación de pull requests

Esto permitió ejecución completamente automática del flujo CI/CD.

9. Resultados de la simulación

Se ejecutaron 5 iteraciones de desarrollo controlado.

Se generaron errores intencionales y correcciones.

Jenkins respondió correctamente a todos los eventos.

Se validó el flujo completo de integración continua.

Se implementó monitoreo en múltiples canales.

10. Conclusiones

La simulación permitió entender el comportamiento real de un pipeline CI/CD.

Jenkins gestiona correctamente estados de error, éxito y advertencia.

El uso de ramas mejora el control de versiones y reduce conflictos.

Las notificaciones en múltiples canales mejoran la supervisión del sistema.

La integración con GitHub permite automatización total del flujo.

11. Recomendaciones

Mantener separación clara entre ramas feature, dev y main.

Automatizar pruebas más profundas en cada commit.

Revisar conflictos antes del merge final.

Usar notificaciones para seguimiento en tiempo real.